

```

require('dotenv').config();
const path = require('path');
const axios = require('axios');
const cron = require('node-cron');
const cheerio = require('cheerio');
const TelegramBot = require('node-telegram-bot-api');
const { TwitterApi } = require('twitter-api-v2');
const Database = require('better-sqlite3');

// — Config —————
const config = {
  sources: {
    telegram: ["@wolf_ofertas", "@achadinhosafiliadosss", "@clubedeofertasss", "@gat
    twitter: ["@xetdaspromocoes", "@achadinhos_dadu", "@achadinhodasho", "@acheinax
  },
  targets: {
    twitterEnabled: true,
    telegramGroups: ["@KingBRVip10"]
  },
  affiliates: {
    mercadolivres: { trackingId: "aler8930883" },
    shopee: { affiliateId: "18325130155" }
  },
  footer: "👉 Mais ofertas: t.me/KingBRVip10"
};

// — Database —————
const db = new Database(path.join(__dirname, 'affiliatebot.db'));
db.pragma('journal_mode = WAL');
db.exec(`
  CREATE TABLE IF NOT EXISTS posts (
    id TEXT NOT NULL, source_type TEXT NOT NULL, source_profile TEXT,
    raw_content TEXT, product_name TEXT, original_link TEXT, image_url TEXT,
    original_price REAL, sale_price REAL, discount INTEGER,
    status TEXT NOT NULL DEFAULT 'pending', affiliate_link TEXT,
    captured_at TEXT NOT NULL, published_at TEXT,
    PRIMARY KEY (id, source_type)
  );
  CREATE TABLE IF NOT EXISTS bot_state (
    chat_id TEXT PRIMARY KEY, state TEXT NOT NULL,
    pending_post_id TEXT, pending_source_type TEXT, updated_at TEXT NOT NULL
  );
`);
console.log('[DB] Banco inicializado.');
```

```

const postExists = (id, t) => !!db.prepare('SELECT 1 FROM posts WHERE id=? AND so

function savePost(p) {
  db.prepare(`
    INSERT OR IGNORE INTO posts
      (id, source_type, source_profile, raw_content, product_name,
        original_link, image_url, original_price, sale_price, discount, captured_a
    VALUES
      (@id, @sourceType, @sourceProfile, @rawContent, @productName,
        @originalLink, @imageUrl, @originalPrice, @salePrice, @discount, @captured
  `).run({
    id:          p.id,
    sourceType:   p.sourceType,
    sourceProfile: p.sourceProfile,
    rawContent:   p.rawContent || null,
    productName:  p.productName || null,
    originalLink: p.originalLink || null,
    imageUrl:     p.imageUrl || null,
    originalPrice: p.originalPrice || null,
    salePrice:    p.salePrice || null,
    discount:     p.discount || null,
    capturedAt:   new Date().toISOString(),
  });
}

const getPost = (id, t) => db.prepare('SELECT * FROM posts WHERE id=? AND source_

function updatePost(id, t, status, extra = {}) {
  db.prepare(`
    UPDATE posts SET
      status          = @status,
      affiliate_link = COALESCE(@afl, affiliate_link),
      published_at    = CASE WHEN @status = 'published' THEN @now ELSE published_a
    WHERE id = @id AND source_type = @t
  `).run({ id, t, status, afl: extra.affiliateLink || null, now: new Date().toISO
}

const getStats = () => ({
  total:    db.prepare("SELECT COUNT(*) as n FROM posts").get().n,
  pending:  db.prepare("SELECT COUNT(*) as n FROM posts WHERE status='pending'")
  published: db.prepare("SELECT COUNT(*) as n FROM posts WHERE status='published'")
  rejected: db.prepare("SELECT COUNT(*) as n FROM posts WHERE status='rejected'")
});

const getBotState = (c) => db.prepare('SELECT * FROM bot_state WHERE cha
const clearState = (c) => db.prepare('DELETE FROM bot_state WHERE chat_

```

```

const setBotState = (c, s, p=null, t=null) =>
  db.prepare(`INSERT INTO bot_state (chat_id,state,pending_post_id,pending_source
    VALUES (@c,@s,@p,@t,@n) ON CONFLICT(chat_id) DO UPDATE SET
    state=excluded.state, pending_post_id=excluded.pending_post_id,
    pending_source_type=excluded.pending_source_type, updated_at=excluded.updated
    .run({ c: String(c), s, p, t, n: new Date().toISOString() });

// ——— Helpers ———
const rnd = (a) => a[Math.floor(Math.random() * a.length)];
const fmtBRL = (v) => v != null ? v.toLocaleString('pt-BR', { style: 'currency',
const sleep = (ms) => new Promise(r => setTimeout(r, ms));

// Helper: get field from post regardless of camelCase or snake_case
const f = (post, camel, snake) => post[camel] !== undefined ? post[camel] : post[

// ——— Link Extractor ———
function extractLinks(text) {
  if (!text) return { hasProductLink: false, firstProductLink: null, source: 'unk

  const ml = text.match(/https?:\/\/\/[^\s<>"]*(?:mercadolivre\.com\.br|mercado
  const shopee = text.match(/https?:\/\/\/[^\s<>"]*(?:shopee\.com\.br|shopee\.ee|s\.

  // Also try to extract any URL if no product link found
  const anyUrl = text.match(/https?:\/\/\/[^\s<>"]+/gi) || [];

  return {
    mlLinks:      ml,
    shopeeLinks:  shopee,
    hasProductLink: ml.length > 0 || shopee.length > 0,
    firstProductLink: ml[0] || shopee[0] || null,
    source:       ml.length > 0 ? 'mercadolivre' : shopee.length > 0 ? 'shope
    anyUrls:      anyUrl,
  };
}

function extractPrices(text) {
  if (!text) return {};
  const prices = [];
  let m;
  const re = /R\$[s?](\d{1,3})(?:[.]\d{3})?(?:[.]\d{2})?)/g;
  while ((m = re.exec(text)) !== null) {
    const v = parseFloat(m[1].replace(/\. /g, '').replace(',', ' '));
    if (!isNaN(v) && v > 0) prices.push(v);
  }
  const dm = text.match(/(\d{1,3})\$*%\$*(?:off|de desconto|OFF)/i);
  const discount = dm ? parseInt(dm[1]) : null;
  if (prices.length >= 2) {

```

```

    const hi = Math.max(...prices), lo = Math.min(...prices);
    return { originalPrice: hi, salePrice: lo, discount: discount || Math.round((
  }
  if (prices.length === 1) return { salePrice: prices[0], discount };
  return { discount };
}

```

```

function extractName(text) {
  if (!text) return null;
  const clean = text.replace(/https?:\/\/\.[^\s]+/g, '').trim();
  for (const line of clean.split('\n').map(l => l.trim()).filter(Boolean)) {
    if (line.length > 8) {
      const n = line.replace(/^[ \p{Emoji}\s]+/u, '').trim();
      if (n.length > 5) return n.slice(0, 120);
    }
  }
  return null;
}

```

```

// ——— Formatter ———
const OPENERS = ['🔥 ACHADO DO DIA', '⚡ CORRE QUE TÁ BARATO', '📌 OFERTA IMPERDÍVEL']
const INTROS = ['Saíndo por só', 'Tá saíndo a', 'Por apenas', 'Você leva por', 'Pode
const DISC_FN = [(d)=>`$${d}% de desconto!`, (d)=>`$${d}% OFF, não é brincadeira!`, (
const CTAS = ['👉 Pega o seu aqui:', '🔗 Link direto:', '👉 Garante o seu agora

```

```

function priceBlock(post) {
  const price = f(post, 'salePrice', 'sale_price');
  if (!price) return '';
  const orig = f(post, 'originalPrice', 'original_price');
  const disc = post.discount;
  let b = `${rnd(INTROS)} ${fmtBRL(price)}`;
  const d = disc || (orig > price ? Math.round(((orig - price) / orig) * 100) : 0
  if (d > 0) b += ` - ${rnd(DISC_FN)(d)}`;
  return b;
}

```

```

function payExtras(raw = '') {
  const l = raw.toLowerCase(); const e = [];
  if (/sem juros|parcelado/.test(l)) e.push('sem juros no cartão');
  if (/frete gr[áa]tis|entrega gr[áa]tis|prime|full/.test(l)) e.push('🚚 frete gr
  return e.length ? ` (${e.join(', ')})` : '';
}

```

```

function formatTelegram(post, link) {
  const name = f(post, 'productName', 'product_name') || '';
  const raw = f(post, 'rawContent', 'raw_content') || '';
  const pb = priceBlock(post);

```

```

    return [rnd(OPENERS)+'!', '', name?`*${name}*`:null, pb?`\n💰 ${pb}${payExtras(raw
        .filter(x => x !== null).join('\n').trim());
}

function formatTwitter(post, link) {
    const name = (f(post, 'productName', 'product_name') || '').slice(0, 55);
    const pb = priceBlock(post).replace(/\*/g, '');
    const suffix = ' | t.me/KingBRVip10!';
    let body = name && pb ? `${rnd(OPENERS)}\n${name}\n${pb}` : name ? `${rnd(OPENE
    if (body.length > 200) body = `${rnd(OPENERS)}\n${name.slice(0, 40)}...`;
    return `${body}\n${rnd(CTAS)} ${link}${suffix}`.trim();
}

function formatPreview(post) {
    const sourceType = f(post, 'sourceType', 'source_type');
    const sourceProfile = f(post, 'sourceProfile', 'source_profile') || '(desconhec
    const productName = f(post, 'productName', 'product_name');
    const originalLink = f(post, 'originalLink', 'original_link');
    const salePrice = f(post, 'salePrice', 'sale_price');
    const src = sourceType === 'shopee' ? '🛒 Shopee' : '🟡 Mercado Livre';
    const price = salePrice ? `\n💰 ${fmtBRL(salePrice)}` + (post.discount ? ` (*${
    return [
        `🆕 *Nova oferta capturada!*`, ``,
        `📌 *Fonte:* ${src} - ${sourceProfile}`,
        productName ? `📦 *Produto:* ${productName}` : null,
        price || null, ``,
        `🔗 *Link original:*`,
        originalLink || '_(não detectado)_', ``,
        `👉_Clique em Aprovar para publicar com seu link de afiliado._`,
    ].filter(Boolean).join('\n');
}

// — Affiliate Links —————
async function generateAffiliateLink(url) {
    if (!url) return null;
    if (/mercadolivre\.com\.br|mercadolibre\.com|meli\.store/i.test(url)) {
        try {
            const r = await axios.get('https://api.mercadolibre.com/link-building', {
                params: { tracking_id: config.affiliates.mercadolivre.trackingId, url },
            });
            if (r.data?.url || r.data?.link) return r.data.url || r.data.link;
        } catch (_) {}
        try {
            const u = new URL(url); u.searchParams.set('tracking_id', config.affiliates
        } catch (_) { return url + '?tracking_id=' + config.affiliates.mercadolivre.t
        }
    }
    if (/shopee\.com\.br|shope\.ee/i.test(url)) {

```

```

    try {
      const u = new URL(url); u.searchParams.set('af_id', config.affiliates.shopee);
    } catch (_) { return url + '?af_id=' + config.affiliates.shopee.affiliateId; }
  }
  return url;
}

```

// — Publishers —————

```
const pubBot = new TelegramBot(process.env.TELEGRAM_BOT_TOKEN);
```

```

async function publishTelegram(text, imageUrl, groups) {
  const results = [];
  for (const g of groups) {
    try {
      if (imageUrl) {
        try { await pubBot.sendPhoto(g, imageUrl, { caption: text, parse_mode: 'Markdown' }); }
        await pubBot.sendMessage(g, text, { parse_mode: 'Markdown', disable_web_page_preview: true });
        results.push({ g, ok: true });
      } catch (e) { results.push({ g, ok: false, error: e.message }); }
    }
  }
  return results;
}

```

```

async function publishTwitter(text) {
  try {
    const client = new TwitterApi({ appKey: process.env.TWITTER_API_KEY, appSecretKey: process.env.TWITTER_API_SECRET });
    const t = await client.v2.tweet(text);
    return { ok: true, id: t.data.id };
  } catch (e) { return { ok: false, error: e.message }; }
}

```

// — Scrapers —————

```

async function scrapeTelegram(channel) {
  const ch = channel.replace('@', '');
  const r = await axios.get(`https://t.me/s/${ch}`, {
    headers: { 'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4399.72 Safari/537.36' },
    timeout: 15000,
  });
  const $ = cheerio.load(r.data);
  const posts = [];

  $('.tgme_widget_message').each((_, el) => {
    const $el = $(el);
    const dataPost = $el.attr('data-post') || '';
    const id = dataPost.split('/').pop();
    if (!id) return;
  });
}

```

```

const raw = $el.find('.tgme_widget_message_text').text().trim();

// Also check for links in anchor tags
const anchorLinks = [];
$el.find('a').each((_, a) => {
  const href = $(a).attr('href');
  if (href) anchorLinks.push(href);
});

const allText = raw + ' ' + anchorLinks.join(' ');
const links = extractLinks(allText);
if (!links.hasProductLink) return;

const prices = extractPrices(raw);
const bgStyle = $el.find('.tgme_widget_message_photo_wrap').attr('style') ||
const imgMatch = bgStyle.match(/url\(['"]?([^"']+)['"]?\)/);

posts.push({
  id,
  sourceType: links.source,
  sourceProfile: `@${ch}`,
  rawContent: raw,
  productName: extractName(raw),
  originalLink: links.firstProductLink,
  imageUrl: imgMatch ? imgMatch[1] : null,
  originalPrice: prices.originalPrice || null,
  salePrice: prices.salePrice || null,
  discount: prices.discount || null,
});
});

return posts;
}

const twitterUserCache = {};
async function scrapeTwitter(username) {
  const handle = username.replace('@', '');
  const client = new TwitterApi(process.env.TWITTER_BEARER_TOKEN);

  if (!twitterUserCache[handle]) {
    const user = await client.readOnly.v2.userByUsername(handle);
    if (!user?.data?.id) return [];
    twitterUserCache[handle] = user.data.id;
  }

  const tl = await client.readOnly.v2.userTimeline(twitterUserCache[handle], {

```

```

    max_results: 10,
    'tweet.fields': ['created_at', 'entities', 'text'],
    expansions: ['attachments.media_keys'],
    'media.fields': ['url', 'preview_image_url'],
  });

const tweets = tl.data?.data || [];
const media = tl.data?.includes?.media || [];
const mediaMap = {};
media.forEach(m => { mediaMap[m.media_key] = m.url || m.preview_image_url; });

return tweets.map(t => {
  // Expand t.co URLs
  const expandedUrls = (t.entities?.urls || []).map(u => u.expanded_url || '').
  const fullText = t.text + ' ' + expandedUrls;
  const links = extractLinks(fullText);
  if (!links.hasProductLink) return null;

  const mediaKeys = t.attachments?.media_keys || [];
  return {
    id: t.id,
    sourceType: links.source,
    sourceProfile: `@${handle}`,
    rawContent: t.text,
    productName: extractName(t.text),
    originalLink: links.firstProductLink,
    imageUrl: mediaKeys[0] ? mediaMap[mediaKeys[0]] : null,
    ...extractPrices(t.text),
  };
}).filter(Boolean);
}

// — Approval Bot —————
const ADMIN = process.env.TELEGRAM_ADMIN_CHAT_ID;
const approvalBot = new TelegramBot(process.env.TELEGRAM_BOT_TOKEN, { polling: tr

async function sendForApproval(post) {
  const text = formatPreview(post);
  const kbd = { inline_keyboard: [[
    { text: '✅ Aprovar', callback_data: `approve:${post.id}:${post.sourceType}` },
    { text: '❌ Rejeitar', callback_data: `reject:${post.id}:${post.sourceType}` },
    { text: '⏮ Pular', callback_data: `skip:${post.id}:${post.sourceType}` },
  ]]];
  try {
    if (post.imageUrl) {
      try { await approvalBot.sendPhoto(ADMIN, post.imageUrl, { caption: text, pa
    }

```



```

    await approvalBot.sendMessage(ADMIN, text, { parse_mode: 'Markdown', reply_ma
  } catch (e) { console.error('[BOT] Erro ao enviar aprovação:', e.message); }
}

approvalBot.on('callback_query', async (q) => {
  const chatId = String(q.message.chat.id);
  const parts = (q.data || '').split(':');
  const action = parts[0];
  await approvalBot.answerCallbackQuery(q.id);

  if (action === 'reject') {
    updatePost(parts[1], parts[2], 'rejected');
    clearState(chatId);
    await approvalBot.sendMessage(chatId, '❌ Post rejeitado.');
```

 Post rejeitado.

```

    return;
  }
  if (action === 'skip') {
    clearState(chatId);
    await approvalBot.sendMessage(chatId, '⏮ Post pulado.');
```

 Post pulado.

```

    return;
  }
  if (action === 'approve') {
    setBotState(chatId, 'waiting_link', parts[1], parts[2]);
    await approvalBot.sendMessage(chatId, '🔗 *Manda seu link de afiliado* (ou `.`
    return;
  }
  if (action === 'ch') {
    await doPublish(chatId, parts[1], parts[2], parts[3].includes('T1'), parts[3]
  }
});

approvalBot.on('message', async (msg) => {
  const chatId = String(msg.chat.id);
  if (chatId !== String(ADMIN)) { if (msg.text === '/start') await approvalBot.se
  if (msg.text?.startsWith('/')) { await handleCmd(msg); return; }

  const state = getBotState(chatId);
  if (!state || state.state !== 'waiting_link') return;

  const afl = msg.text?.trim() === '.' ? '' : msg.text?.trim();
  updatePost(state.pending_post_id, state.pending_source_type, 'approved', { affi
  setBotState(chatId, 'waiting_channels', state.pending_post_id, state.pending_so

  const hasT = config.targets.twitterEnabled;
  const hasG = config.targets.telegramGroups.length > 0;
  const pid = state.pending_post_id;
  const ptype = state.pending_source_type;

```

```

const btns = [];
if (hasT && hasG) {
  btns.push([ { text: 'X Twitter + ➔ Telegram', callback_data: `ch:${pid}:${ptyp}`
  btns.push([ { text: 'X Só Twitter', callback_data: `ch:${pid}:${ptype}:T1G0` }
} else if (hasT) {
  btns.push([ { text: 'X Publicar no Twitter', callback_data: `ch:${pid}:${ptype}`
} else {
  btns.push([ { text: '➔ Publicar no Telegram', callback_data: `ch:${pid}:${ptyp}`
}
await approvalBot.sendMessage(chatId, '📢 *Onde publicar?*', { parse_mode: 'Mar
});

```

```

approvalBot.on('polling_error', (e) => console.error('[BOT] Polling error:', e.me

```

```

async function doPublish(chatId, postId, sourceType, useTwitter, useTelegram) {
  clearState(chatId);
  const post = getPost(postId, sourceType);
  if (!post) { await approvalBot.sendMessage(chatId, '⚠️ Post não encontrado.');
```

```

  const rawLink = post.affiliate_link || post.original_link;
  const link = await generateAffiliateLink(rawLink);
  let ok = 0; const errs = [];

  if (useTelegram && config.targets.telegramGroups.length > 0) {
    const text = formatTelegram(post, link);
    const res = await publishTelegram(text, post.image_url, config.targets.teleg
    res.forEach(r => { if (r.ok) ok++; else errs.push(`Telegram ${r.g}: ${r.error`
  }
  if (useTwitter && config.targets.twitterEnabled) {
    const text = formatTwitter(post, link);
    const res = await publishTwitter(text);
    if (res.ok) ok++; else errs.push(`Twitter: ${res.error}`);
  }

  updatePost(postId, sourceType, 'published', { affiliateLink: link });
  const msg = ok > 0
    ? `✅ *Publicado!* Enviado para ${ok} canal(is).${errs.length ? '\n⚠️ ' + err
    : `❌ *Falha ao publicar.*\n${errs.join('\n')}`;
  await approvalBot.sendMessage(chatId, msg, { parse_mode: 'Markdown' });
}

```

```

async function handleCmd(msg) {
  const chatId = String(msg.chat.id);
  const cmd = msg.text.split(' ')[0].toLowerCase();
  if (cmd === '/start' || cmd === '/help') {
    await approvalBot.sendMessage(chatId, '🤖 *AffiliateBot*\n\n`/status` - Estat
  } else if (cmd === '/status') {

```

```

    const s = getStats();
    await approvalBot.sendMessage(chatId, `🇧🇷 *Stats*\n\n📦 Total: ${s.total}\n⌚`);
  } else if (cmd === '/config') {
    await approvalBot.sendMessage(chatId, `⚙️ *Config atual*\n\n📡 Telegram: ${cc}`);
  } else if (cmd === '/cancel') {
    clearState(chatId);
    await approvalBot.sendMessage(chatId, '🔙 Cancelado.');
```

```

  }
}

// — Main loop —
async function checkPosts() {
  const ts = new Date().toLocaleTimeString('pt-BR');
  console.log(`\n[${ts}] 🔍 Verificando novos posts...`);

  for (const ch of config.sources.telegram) {
    try {
      const posts = await scrapeTelegram(ch);
      let n = 0;
      for (const p of posts) {
        if (postExists(p.id, p.sourceType)) continue;
        savePost(p);
        await sendForApproval(p);
        n++;
        await sleep(1500);
      }
      console.log(` [TG] ${ch} → ${n} novo(s)`);
    } catch (e) { console.error(` [TG] Erro em ${ch}:`, e.message); }
  }

  if (process.env.TWITTER_BEARER_TOKEN) {
    for (const u of config.sources.twitter) {
      try {
        const tweets = await scrapeTwitter(u);
        let n = 0;
        for (const t of tweets) {
          if (postExists(t.id, t.sourceType)) continue;
          savePost(t);
          await sendForApproval(t);
          n++;
          await sleep(1500);
        }
        console.log(` [TW] ${u} → ${n} novo(s)`);
      } catch (e) { console.error(` [TW] Erro em ${u}:`, e.message); }
    }
  }
}

```

```

async function main() {
  const missing = ['TELEGRAM_BOT_TOKEN', 'TELEGRAM_ADMIN_CHAT_ID'].filter(k => !p
  if (missing.length) { console.error('❌ Variáveis ausentes:', missing.join(', '

  console.log(`
  ||  AffiliateBot iniciado! ||
  ||  Telegram : ${String(config.sources.telegram.length + ' canais').padEnd(26)} ||
  ||  Twitter : ${String(config.sources.twitter.length + ' perfis').padEnd(26)} ||
  ||  Publicando em @KingBRVip10 + X ||
  || `);

  await checkPosts();
  const min = parseInt(process.env.CHECK_INTERVAL_MINUTES) || 15;
  cron.schedule(`*/${min} * * * *`, checkPosts);
}

main().catch(e => { console.error('💥 Erro fatal:', e.message); process.exit(1);

```